PROYECTO

CNN Visualizer

How AI Sees Your Numbers

Visualizador interactivo de redes neuronales convolucionales en el browser.

100%	TF.js	Canvas	Paso a
Browser sin backend	MNIST preentrenado	Draw cualquier número	paso animado

01 Visión General

El usuario dibuja un número (0–9) en un canvas interactivo. La app procesa ese número en tiempo real usando TensorFlow.js y muestra, paso a paso, qué ocurre dentro de una red neuronal convolucional — desde los píxeles de entrada hasta la predicción final.

La experiencia completa:

```
Dibujas "7"
    ↓
Input Layer  →  28×28 píxeles en escala de grises
    ↓
Conv2D #1    →  kernel 3×3 se desliza, detecta bordes
    ↓
MaxPooling   →  reduce a la mitad, se queda con lo importante
    ↓
Conv2D #2    →  detecta patrones más complejos
    ↓
Flatten      →  convierte el mapa en un vector
    ↓
Dense        →  128 neuronas deciden
    ↓
Output       →  "Es un 7 — confianza: 98.3%"
```

02 Stack Tecnológico

CAPA	TECNOLOGÍA	PROPÓSITO
ML	TensorFlow.js	Modelo MNIST corriendo 100% en el browser

CAPA	TECNOLOGÍA	PROPÓSITO
3D/Anim	Three.js + GSAP	Animaciones de capas y kernel en tiempo real
UI	Vanilla TypeScript	Sin framework, máximo control y performance
Build	Vite	Dev server ultra rápido, HMR instantáneo
Hosting	GitHub Pages	Gratis, deploy automático desde el repo
Assets	Canvas API	Para el área de dibujo interactivo

03 Arquitectura del Proyecto

cnn-visualizer/	Raíz del proyecto
├── src/	
│ ├── main.ts	Entry point, inicializa todo
│ ├── canvas/	
│ │ ├── DrawCanvas.ts	Lógica de dibujo + normalización 28×28
│ │ └── PixelGrid.ts	Visualización del grid de entrada
│ ├── nn/	
│ │ ├── model.ts	Carga modelo MNIST + extrae activaciones
│ │ └── layers.ts	Definición de capas + metadata visual
│ ├── viz/	
│ │ ├── LayerRenderer.ts	Renderiza cada capa en Three.js
│ │ ├── KernelAnim.ts	Animación del kernel deslizándose
│ │ └── FeatureMap.ts	Construye el feature map paso a paso
│ ├── ui/	
│ │ ├── StepPanel.ts	Panel lateral con explicación por capa
│ │ └── OutputBar.ts	Barras de confianza 0–9
│ └── shaders/	

│ ├── activation.vert	Vertex shader para efecto glow
│ └── activation.frag	Fragment shader — valor → color
├── public/	
│ └── model/	Modelo MNIST en formato TF.js (JSON + weights)
├── index.html	HTML base
├── vite.config.ts	Config de Vite
└── package.json	Dependencias

04 Fases de Desarrollo

FASE 1 Canvas + Modelo Base · *3–5 días*

- → Canvas 280×280 con dibujo libre (mouse y touch)
- → Downsample automático a 28×28 con normalización
- → Cargar modelo MNIST preentrenado con tf.loadLayersModel()
- → Primera predicción funcionando — output en consola
- → Grid visual de 28×28 celdas que se colorea con tu dibujo

FASE 2 Visualización de Capas · *1 semana*

- → Renderizar cada capa como un plano de cubitos en Three.js
- → Color del cubito mapeado desde el valor de activación (negro → cyan → blanco)
- → Animar la ola de activación que viaja de capa en capa con delay
- → Panel lateral con nombre y descripción de cada capa
- → Cámara que hace paneo suave mientras procesa

FASE 3 Animación del Kernel · *1 semana*

- → Kernel 3×3 visible como un highlight naranja sobre el input grid
- → Animación frame a frame del kernel deslizándose
- → Feature map construyéndose en paralelo, píxel por píxel
- → Modo "paso a paso" y modo "automático" con slider de velocidad
- → Shaders GLSL para efecto de glow en valores altos de activación

FASE 4 Output + Polish · *3–4 días*

- → Barras de confianza animadas para cada dígito 0–9
- → Highlight del dígito ganador con efecto de pulso

→ Modo oscuro ultra limpio, tipografía monoespaciada
→ Responsive — funciona en móvil y desktop
→ Deploy en GitHub Pages con GitHub Actions

05 Lo Que Aprenderás

CONCEPTO	POR QUÉ IMPORTA
Extraer activaciones intermedias de TF.js	Entiendes qué aprende cada capa, no solo el output
Shaders GLSL (vertex + fragment)	El salto a GPU programming — mapear números a color/glow
Animaciones procedurales con GSAP + Three.js	Sincronizar estado de ML con geometría 3D
Canvas API a bajo nivel	Dibujo, composición de píxeles, downsample manual
Cómo funciona MNIST por dentro	La CNN más famosa del mundo, entendida de verdad
Deploy automático con GitHub Actions	CI/CD básico — empujas código, se despliega solo

06 Setup Inicial

Crear el proyecto

```
npm create vite@latest cnn-visualizer -- --template vanilla-ts
cd cnn-visualizer
npm install
npm install @tensorflow/tfjs three gsap
npm run dev
```

Descargar el modelo MNIST preentrenado

```
# En /public/model/ necesitas estos archivos:
#   model.json        → arquitectura
#   group1-shard1of1.bin → pesos entrenados
#
# Fuente oficial TF.js:
# https://storage.googleapis.com/tfjs-
models/tfjs/mnist_transfer_cnn_v1/model.json
```

Verificar que el modelo carga

```
import * as tf from '@tensorflow/tfjs';

const model = await tf.loadLayersModel('/model/model.json');
console.log(model.summary()); // deberías ver las capas

// Extraer activaciones intermedias:
const layerNames = model.layers.map(l => l.name);
const intermediateModel = tf.model({
  inputs: model.inputs,
  outputs: model.layers.map(l => l.output)
});
```

07 Recursos Clave

RECURSO	URL
TF.js Docs	https://js.tensorflow.org/api/latest/
Modelo MNIST	https://github.com/tensorflow/tfjs-examples/tree/master/mnist
Three.js Docs	https://threejs.org/docs/
GSAP Docs	https://greensock.com/docs/
Vite Config	https://vitejs.dev/config/
CNN Explicado	https://cs231n.github.io/convolutional-networks/
GitHub Pages Deploy	https://vitejs.dev/guide/static-deploy.html#github-pages
GLSL Tutorial	https://thebookofshaders.com/